

Week8

Task1:

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#define MAX 100
int main(){
    char input[500];
    char *args[MAX];
    int count=0;
    int redirect=0;
    int append=0;
    int merge=0;
    char *outfile=NULL;
    printf("Enter command: ");
    fgets(input,sizeof(input),stdin);
    input[strcspn(input,"\n")]='\0';
    char *token=strtok(input," ");
    while(token!=NULL){
        if(strcmp(token,>")==0){
            redirect=1;
            append=0;
            token=strtok(NULL," ");
            if(token!=NULL)
                outfile=token;
        }
        else if(strcmp(token,>>")==0){
            redirect=1;
            append=1;
            token=strtok(NULL," ");
            if(token!=NULL)
                outfile=token;
        }
        else if(strcmp(token,"2>&1")==0){
            merge=1;
        }else{
            args[count++]=token;
        }
        token=strtok(NULL," ");
    }
}
```

GNU nano 7.2

```
args[count]=NULL;
if(count==0){
    printf("Error: No command entered.\n");
    return 1;
}
if(redirect && outfile==NULL){
    printf("Error: Output file missing.\n");
    return 1;
}
pid_t pid=fork();
if(pid<0){
    perror("fork");
    return 1;
}
if(pid==0){
    if(redirect){
        int fd;
        if(append)
            fd=open(outfile,O_WRONLY|O_CREAT|O_APPEND,0644);
        else
            fd=open(outfile,O_WRONLY|O_CREAT|O_TRUNC,0644);

        if(fd<0){
            perror("open");
            exit(1);
        }

        if(dup2(fd,STDOUT_FILENO)<0){
            perror("dup2");
            close(fd);
            exit(1);
        }
        if(merge){
            if(dup2(STDOUT_FILENO,STDERR_FILENO)<0){
                perror("dup2");
                close(fd);
                exit(1);
            }
        }
        close(fd);
    }
    execvp(args[0],args);
    perror("execvp");
    exit(1);
}
else{
    int status;
    waitpid(pid,&status,0);
    if(WIFEXITED(status))
        printf("Command completed with exit status %d\n",
            WEXITSTATUS(status));
    else
        printf("Command terminated abnormally.\n");
}
return 0;
}
```

```
    }
    close(fd);
}
execvp(args[0],args);
perror("execvp");
exit(1);
}
else{
    int status;
    waitpid(pid,&status,0);
    if(WIFEXITED(status))
        printf("Command completed with exit status %d\n",
            WEXITSTATUS(status));
    else
        printf("Command terminated abnormally.\n");
}
return 0;
}
```

Output:

```
root@DESKTOP-VMN932K:~# ./merge
Enter command: ls wrongfile > output.txt 2>&1
Command completed with exit status 2
root@DESKTOP-VMN932K:~#
```

Task2:

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_LINE 1024
#define MAX_ARGS 100
int main() {
    char input[MAX_LINE];
    printf("Mini Shell Parser\n");
    while (1) {
        printf("shell> ");
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;
        printf("\nInput Command : %s\n", input);
        if (strchr(input, '|'))
            printf("Contains Pipe(s)\n");
        if (strchr(input, '<'))
            printf("Contains Input Redirection\n");
        if (strchr(input, '>'))
            printf("Contains Output Redirection\n");
        printf("\nParsed Tokens:\n");
        char *token = strtok(input, " ");
        int count = 0;
        while (token != NULL) {
            printf("Token %d : %s\n", ++count, token);
            token = strtok(NULL, " ");
        }
        printf("\nExecution Plan:\n");
        printf("1. Validate Syntax\n");
        printf("2. Parse Command\n");
        printf("3. Resolve Pipes/Redirections\n");
        printf("4. Determine Execution Order\n");
        printf("5. Execute Commands\n");
    }
    return 0;
}
```

Output:

```
root@DESKTOP-VMN932K:~# ./validate
Mini Shell Parser
shell> ls

Input Command : ls

Parsed Tokens:
Token 1 : ls

Execution Plan:
1. Validate Syntax
2. Parse Command
3. Resolve Pipes/Redirections
4. Determine Execution Order
5. Execute Commands
shell> pwd

Input Command : pwd

Parsed Tokens:
Token 1 : pwd

Execution Plan:
1. Validate Syntax
2. Parse Command
3. Resolve Pipes/Redirections
4. Determine Execution Order
5. Execute Commands
shell> sort < input.txt > output.txt

Input Command : sort < input.txt > output.txt
Contains Input Redirection
Contains Output Redirection

Parsed Tokens:
Token 1 : sort
Token 2 : <
Token 3 : input.txt
Token 4 : >
Token 5 : output.txt

Execution Plan:
1. Validate Syntax
```

Task3:

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
int main() {
    char command[100];
    while (1) {
        printf("shell> ");
        if (fgets(command, sizeof(command), stdin) == NULL)
            break;
        command[strcspn(command, "\n")] = '\0';
        if (strcmp(command, "exit") == 0)
            break;
        int background = 0;
        int len = strlen(command);
        if (len > 0 && command[len - 1] == '&'){
            background = 1;
            command[len - 1] = '\0'; }
        pid_t pid = fork();
        if (pid < 0){
            perror("fork");
            continue; }
        if (pid == 0){
            execlp(command, command, NULL);
            perror("Command not found");
            exit(1);
        } else {
            if (background){
                printf("Background Job Started\n");
                printf("PID: %d\n", pid); }
            else {
                waitpid(pid, NULL, 0);
                printf("Foreground Job Completed\n"); }
        }
        while (waitpid(-1, NULL, WNOHANG) > 0)
            printf("A Background Job Finished.\n");
    }
    return 0;
}
```

Output:

```
root@DESKTOP-VMN932K:~# ./launch
shell> pwd
/root
Foreground Job Completed
shell> date
Thu Sep 24 09:21:43 UTC 2026
Foreground Job Completed
shell> hostname
DESKTOP-VMN932K
Foreground Job Completed
shell> exit
root@DESKTOP-VMN932K:~#
```

Task4:

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```
#include <stdio.h>
#include <string.h>
struct Job {
    int id, group, size;
    char status[20];
};
int main() {
    struct Job j[10];
    int n = 0, choice, id, i;
    do {
        printf("\n1.Create Job");
        printf("\n2.Add Process Group");
        printf("\n3.Update Status");
        printf("\n4.Remove Completed Job");
        printf("\n5.Display Jobs");
        printf("\n6.Validate Jobs");
        printf("\n7.Exit");
        printf("\nEnter choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                printf("Enter Job ID: ");
                scanf("%d", &j[n].id);
                printf("Enter Group ID: ");
                scanf("%d", &j[n].group);
                printf("Enter Group Size: ");
                scanf("%d", &j[n].size);
                strcpy(j[n].status, "Pending");
                n++;
                break;
            case 2:
                printf("Enter Job ID: ");
                scanf("%d", &id);
                for(i = 0; i < n; i++)
                    if(j[i].id == id)
                        printf("Group: %d, Size: %d\n",
                            j[i].group, j[i].size);
                break;
            case 3:
                printf("Enter Job ID: ");
```

```

        case 3:
            printf("Enter Job ID: ");
            scanf("%d", &id);
            for(i = 0; i < n; i++)
                if(j[i].id == id)
                    strcpy(j[i].status, "Completed");
            break;
        case 4:
            for(i = 0; i < n; i++) {
                if(strcmp(j[i].status, "Completed") == 0) {
                    j[i] = j[n - 1];
                    n--;
                    i--;
                }
            }
            printf("Completed jobs removed.\n");
            break;
        case 5:
            for(i = 0; i < n; i++)
                printf("Job %d | Group %d | Size %d | %s\n",
                    j[i].id, j[i].group,
                    j[i].size, j[i].status);
            break;
        case 6:
            for(i = 0; i < n; i++)
                if(j[i].id <= 0 || j[i].size <= 0)
                    printf("Invalid Job %d\n", j[i].id);
            printf("Validation completed.\n");
            break;
    }
} while(choice != 7);
return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./jobtable

1.Create Job
2.Add Process Group
3.Update Status
4.Remove Completed Job
5.Display Jobs
6.Validate Jobs
7.Exit
Enter choice: 1
Enter Job ID: 1001
Enter Group ID: 1001
Enter Group Size: 2

1.Create Job
2.Add Process Group
3.Update Status
4.Remove Completed Job
5.Display Jobs
6.Validate Jobs
7.Exit
Enter choice: █

```